



Principal Component Analysis of FAANG Stock Price Data

Chetan Babu Mahendiran

For the module STAT9014_31544 as part of the

**Masters in Science in Data Science and Analytics, Department of
Mathematics**

Date - 24.04.2026

Table of Contents

1. Introduction.....	3
1.1 Research Question	3
1.2 Analytical Objective.....	3
1.3 Why Multivariate Analysis and PCA	4
2. Data and Preprocessing.....	5
2.1 Dataset Overview	5
2.2 Missing Values.....	5
2.3 Outlier Detection.....	5
3. Methodology	6
3.1 Suitability Assessment - Correlation and Formal Tests	6
3.2 Bartlett's Test of Sphericity and KMO	7
3.3 Standardisation	7
3.4 PCA Application and Component Selection.....	7
3.4.1 Scree Plot and Variance Explained	7
3.4.2 Component Selection	8
4. Results and Interpretation	9
4.1 Loading Plots	9
4.2 PC1 - Market Price Level Factor (63.6%)	9
4.3 PC2 - Momentum / Oscillator Factor (14.1%)	9
4.4 PC3 - Volume / Volatility Factor (6.6%)	10
4.5 PC4 - Residual Return Factor (6.3%)	10
4.6 Score Plots and Biplot Interpretation	10
5. Comparison with Factor Analysis	12
6. Sensitivity Analysis	13
7. Critical Discussion.....	14
8. Conclusion	15
9. References.....	16
Appendix	17

1. Introduction

This report presents a multivariate analysis of FAANG stock price data, specifically the companies Apple (AAPL), Amazon (AMZN), Google (GOOGL), Meta (META) and Microsoft (MSFT) based on Principal Component Analysis(PCA). It contains 14,964 daily observations and 16 numeric features: raw price variables (Open, High, Low, Close), trading volume; technical moving-average indicators (SMA_7, SMA_21, EMA_12, EMA_26); the Bollinger Bands; momentum oscillators (RSI_14 MACD MACD_Signal); and volatility measures (Daily_Return Volatility_7d).

All 16 features are calculated from a single price series, suggesting they will be highly collinear with one another, however this justifies the need for dimensionality reduction. If we treated them independently, it will yield redundant and contradictory results. PCA gives us a principled way of collapsing this redundancy down to just a few orthogonal components, while still capturing the maximal variance. The follow up analysis is composed of the following pipeline: data preprocessing, suitability test(s), standardisation, PCA selected components loading and score plot interpretation; comparison with Factor Analysis (FA) including sensitivity analysis. All calculations were performed in Python with scikit-learn[1].

1.1 Research Question

Can PCA find a few interpretable latent dimensions in FAANG stock technical indicators that match existing financial theory (i.e., price trend, momentum and volatility) consistently across distinct analytic methods?

1.2 Analytical Objective

The objective is to reduce 16 collinear technical features into a compact set of meaningful components, interpret those components in financial terms, and validate findings through FA comparison and sensitivity testing. This addresses a genuine practical problem: financial datasets are high-dimensional by design, and understanding which orthogonal dimensions drive variation is valuable for feature selection, risk decomposition, and portfolio analytics.

1.3 Why Multivariate Analysis and PCA

Multivariate analysis is appropriate because the 16 features are not independent, they share a common underlying price series and carry heavily overlapping information. Analysing them individually would miss the shared covariance structure and lead to redundant outcomes.

PCA was chosen over alternatives for two reasons. First, there is no labelled class variable to supervise group separation, ruling out Linear Discriminant Analysis (LDA). Second, the goal is data compression and maximum variance retention rather than modelling shared factor loadings the focus of Factor Analysis. PCA is the natural fit when the aim is to find the fewest dimensions that preserve as much of the original information as possible [4]. Factor Analysis is nonetheless applied as a methodological extension to cross-validate the component structure from a complementary modelling perspective.

2. Data and Preprocessing

2.1 Dataset Overview

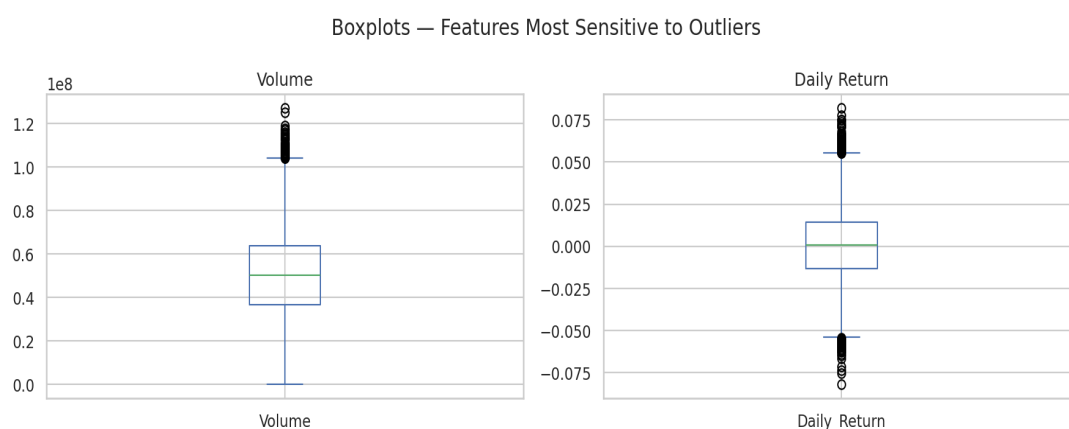
The raw dataset contains 14,964 rows and 19 columns spanning multiple years of daily trading data for five FAANG stocks. The target variable `Next_Day_Close` was excluded from the analysis because PCA is an unsupervised technique, including a future-looking variable would introduce information leakage and distort the component structure. The categorical columns `Date` and `Ticker` were also removed, leaving 16 numeric features.

2.2 Missing Values

A missing Values check verified that all 16 of the numeric features were complete (zero nulls). Excluded columns were dropped, so no imputation was required, and the dataset was used in full.

2.3 Outlier Detection

Outlier detection (threshold $|Z| > 3$) based on z-score identified 1,115 rows (~7.45%) at least one extreme value. Features which had the greatest effect were `Bollinger_Upper` (331 outliers), `EMA_26` (328) and `Volume` (327). These observations were retained. In the case of financial data, those extreme values usually relate to real market phenomena e.g., days with lots of trading volume or earnings announcements or market crashes. Removing them would discard economically relevant information.



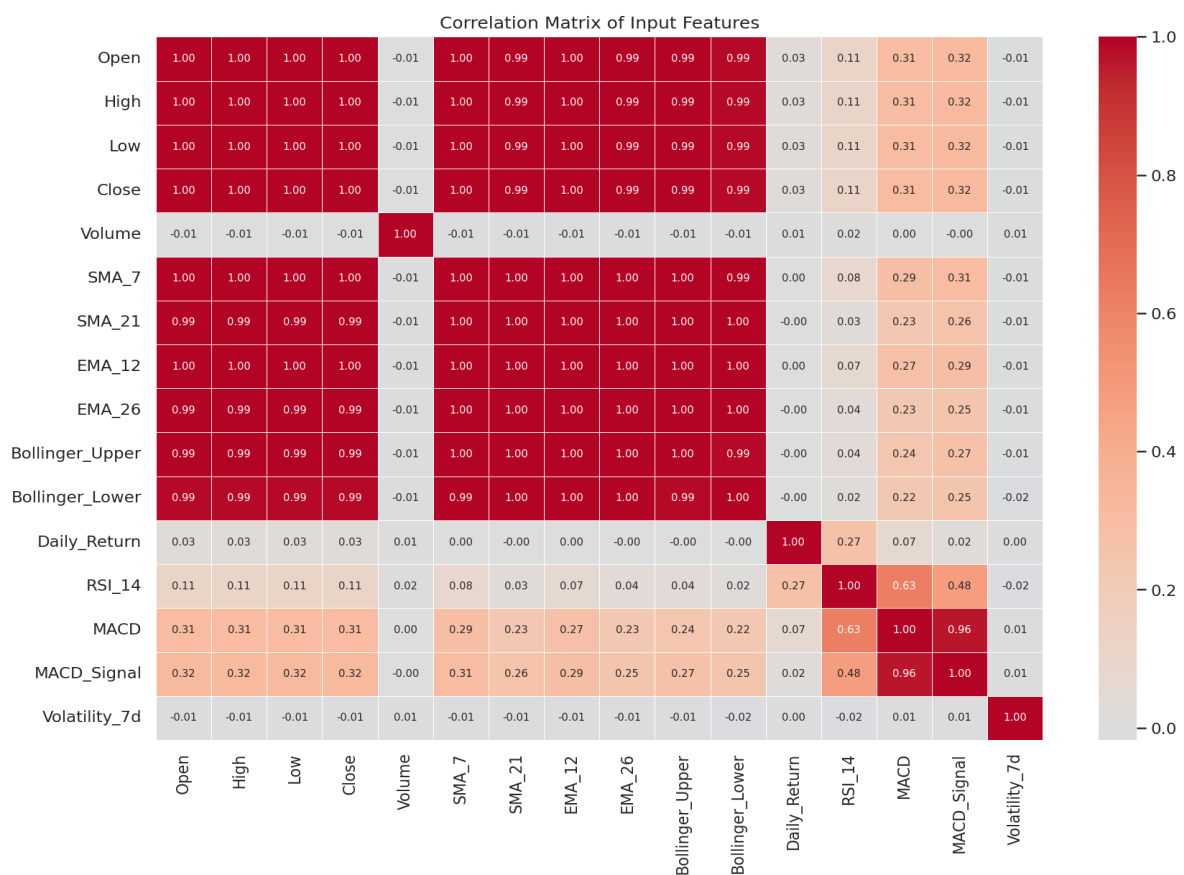
Boxplots of `Volume` and `Daily_Return` the two features most affected by extreme values. Outliers were retained as they represent genuine market events.

3. Methodology

The methodology comprises four stages - assessing suitability for PCA by confirming the correlation structure and formal statistical tests, standardising all features, applying PCA with component selection guided by three criteria, and validating findings through Factor Analysis comparison and sensitivity analysis. All analyses were implemented in Python using scikit-learn [1].

3.1 Suitability Assessment - Correlation and Formal Tests

The correlation heatmap below reveals strong multicollinearity among price-level features (Open, High, Low, Close, SMA_7, SMA_21, EMA_12, EMA_26, Bollinger_Upper, Bollinger_Lower) all near perfectly correlated ($|r| > 0.95$). This high correlation is precisely what makes PCA suitable here the redundant price information can be compressed into a single principal component and reduce the feature space significantly.



Correlation matrix of 16 input features. Near-perfect correlations among price-level variables (top-left block) justify dimensionality reduction via PCA.

3.2 Bartlett's Test of Sphericity and KMO

Bartlett's test [2] checks whether the correlation matrix differs significantly from an identity matrix. An identity matrix would mean all variables are uncorrelated, making PCA meaningless. With the full 16-feature set, the test returned NaN due to a near-zero determinant caused by severe multicollinearity. To resolve this, features with $|r| > 0.95$ were removed for this test only. The reduced set produced a statistically significant result ($p < 0.001$), confirming PCA is appropriate. A KMO statistic of 0.839 was also obtained, falling in the "Meritorious" range (≥ 0.8), confirming that latent factors genuinely exist in the data [3].

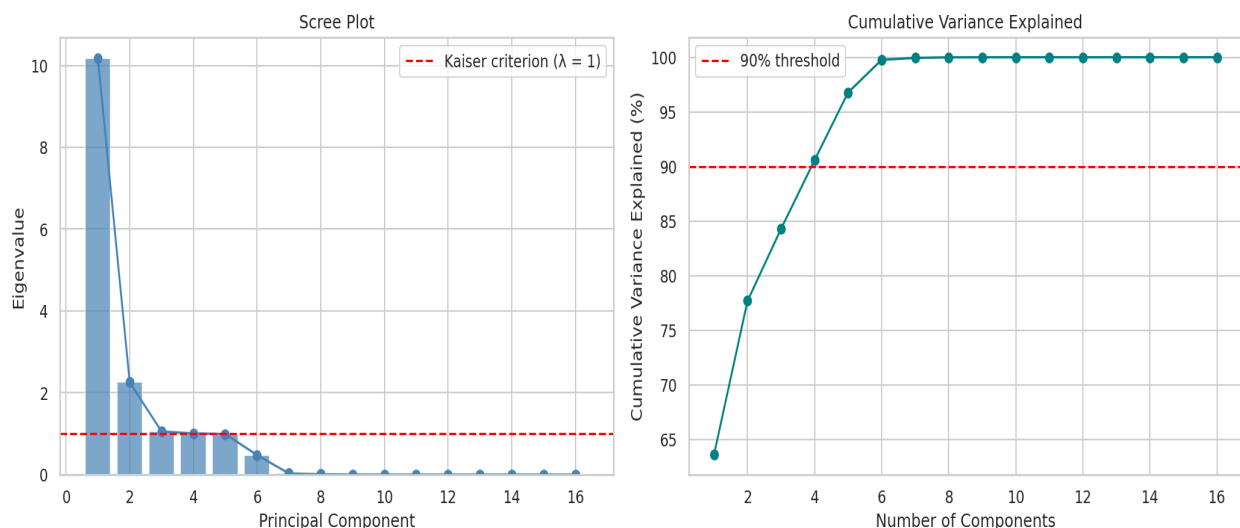
3.3 Standardisation

All features were standardised using z-score normalisation (zero mean, unit variance) before applying PCA. This is essential because PCA is sensitive to scale, Volume - measured in hundreds of millions - would otherwise dominate the principal components by magnitude alone rather than by informational content. Standardisation ensures all 16 features contribute equally to the PCA computation [4].

3.4 PCA Application and Component Selection

3.4.1 Scree Plot and Variance Explained

A full PCA was first fitted to determine how variance distributes across all 16 components. The scree plot and cumulative variance plot below guide component selection.



Scree plot (left) showing eigenvalue decay with Kaiser criterion at $\lambda = 1$, and cumulative variance plot (right) with 90% threshold marked. Both support retaining 4 components.

PC1 alone accounts for 63.6% of total variance, reflecting the dominant influence of strongly correlated price-level features. PC2 adds a further 14.1%, PC3 and PC4 each contribute approximately 6.6% and 6.3% respectively. The remaining 12 components together explain less than 9% of variance.

3.4.2 Component Selection

Three criteria were applied to determine the number of retained components:

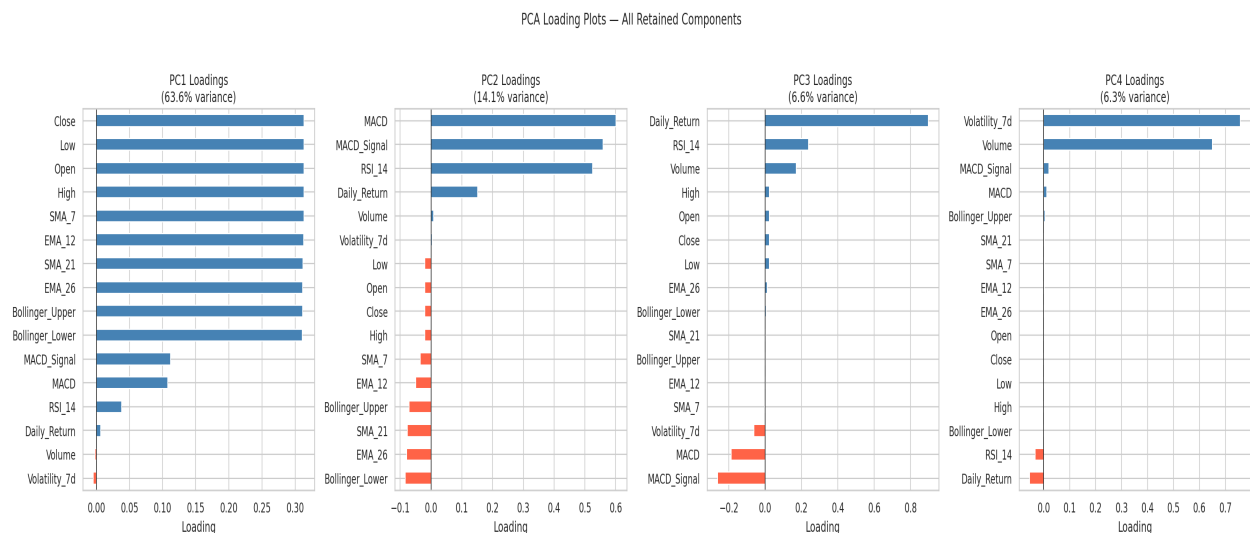
- Kaiser criterion: retain components with eigenvalue > 1 - result : 4 components.
- 90% cumulative variance threshold: retain the minimum set reaching 90% - result : 4 components.
- Scree plot: the elbow in the eigenvalue decay curve also indicates 4 components.

All three criteria agree unanimously. Four components were retained, collectively explaining 90.6% of total variance, with a reconstruction MSE of 0.069.

4. Results and Interpretation

4.1 Loading Plots

Loading plots show the contribution of each original feature to each principal component. Positive loadings (blue) indicate a positive relationship with the component direction; negative loadings (red) indicate an inverse relationship.



Loading plots for all four retained principal components. PC1 shows uniform positive loadings on price features; PC2 is driven by MACD and RSI; PC3 by Volume and Volatility; PC4 by Daily_Return.

4.2 PC1 - Market Price Level Factor (63.6%)

PC1 loads almost equally and positively on all price-related features: Open, High, Low, Close, SMA_7, SMA_21, EMA_12, EMA_26, Bollinger_Upper, and Bollinger_Lower (all loadings ≈ 0.31). Volume loads negatively (-0.14). This component represents the overall price level of a stock. High positive PC1 scores correspond to periods of elevated stock prices. The dominance of this component is expected given that all price features are derived from the same underlying OHLC data and move together with the long-term upward trend common to all FAANG stocks.

4.3 PC2 - Momentum / Oscillator Factor (14.1%)

PC2 is driven primarily by MACD (0.601), MACD_Signal (0.567), and RSI_14 (0.474). These are momentum oscillators that measure the speed and direction of price changes relative to recent history. PC2 captures short-term directional momentum, distinguishing overbought

conditions (high RSI, positive MACD crossover) from oversold regimes [5]. This component provides a signal orthogonal to the raw price level captured by PC1.

4.4 PC3 - Volume / Volatility Factor (6.6%)

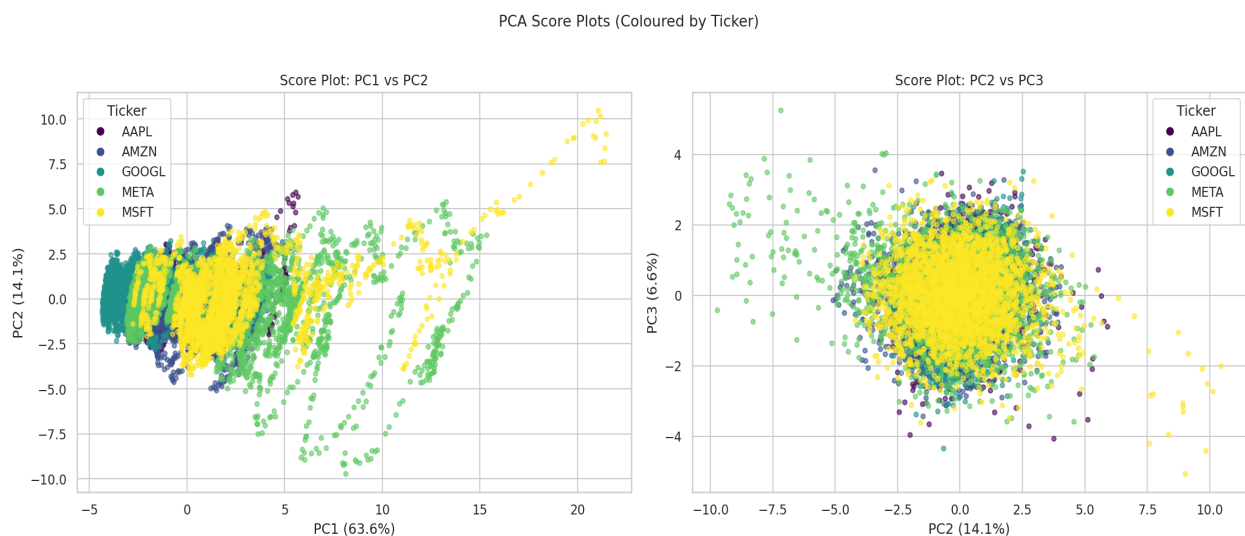
PC3 loads most strongly on Volatility_7d (0.708) and Volume (0.640). This component separates high-activity, volatile trading sessions from calm low-volume periods. Extreme positive scores correspond to days with large price swings and elevated trading activity typically associated with earnings releases or broader market events.

4.5 PC4 - Residual Return Factor (6.3%)

PC4 is primarily influenced by Daily_Return and residual variation in RSI_14 and MACD not captured by PC2. It captures idiosyncratic day-to-day price changes not explained by longer-term trend, momentum, or volatility factors in the first three components.

4.6 Score Plots and Biplot Interpretation

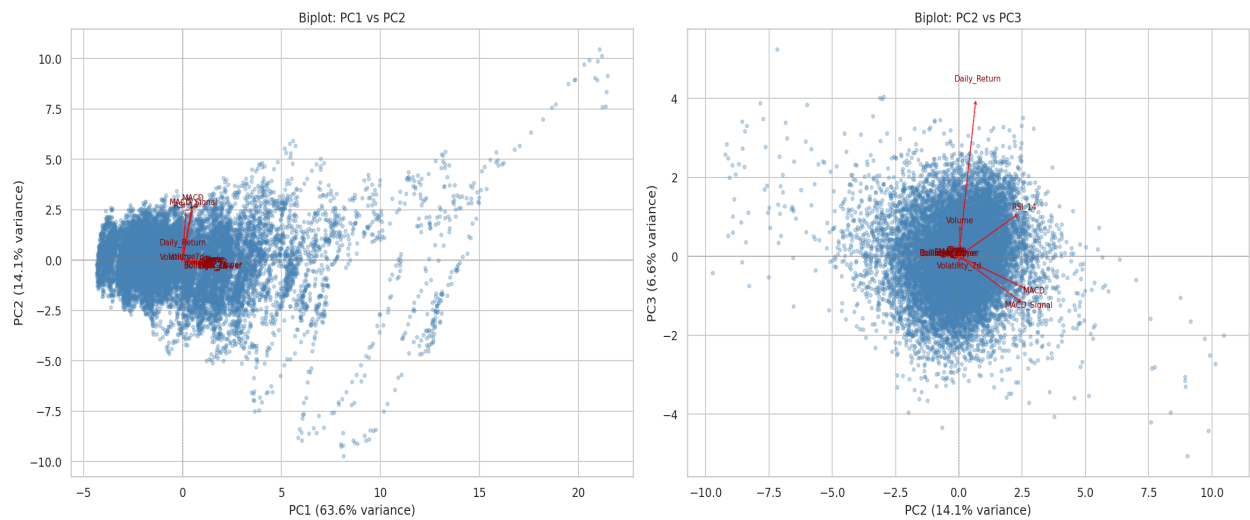
Score plots project all 14,964 observations onto pairs of principal components, coloured by stock ticker. Biplots overlay the original feature loading vectors (arrows) onto the score space.



PCA score plots. Left: PC1 vs PC2 - stocks separate along the PC1 (price level) axis. Right: PC2 vs PC3 - volume/volatility variation distinguishes stocks further.

The PC1 vs PC2 plot shows clear separation of stocks along the PC1 axis, with high-priced stocks scoring higher on PC1. The PC2 axis separates stocks by recent momentum. The PC2 vs PC3 plot reveals how Volume and Volatility interact with momentum, with some tickers showing notably higher PC3 variation during earnings periods.

PCA Biplots

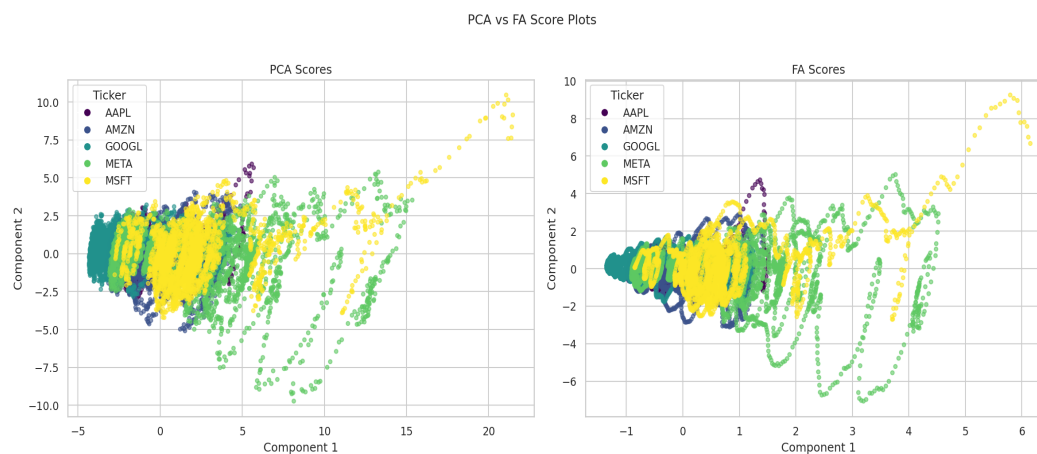


PCA biplots overlaying loading vectors on the score space. Price-feature arrows cluster along the PC1 axis; MACD and RSI arrows project strongly along PC2; Volume and Volatility arrows align with PC3.

The biplots confirm the loading analysis visually. Price-feature arrows cluster tightly along the horizontal PC1 axis, confirming PC1 is a price-level index. MACD and RSI_14 arrows project along PC2, consistent with the momentum interpretation. Volume and Volatility_7d arrows point along PC3, reinforcing the volatility interpretation.

5. Comparison with Factor Analysis

Factor Analysis (FA) was applied with four components for comparison [6]. Unlike PCA, which maximises total variance, FA models only shared (common) variance, treating unique variance as noise. This makes FA more appropriate when the goal is uncovering latent constructs, while PCA is better suited for data compression.



PCA vs Factor Analysis score plots (Component 1 vs Component 2), coloured by ticker. Both methods produce qualitatively similar stock separation patterns, confirming robustness of the latent structure.

FA loadings for Factor 1 are much larger in magnitude (near 1.0 for all price features) compared to PCA loadings (≈ 0.311), because FA does not apply the orthonormality constraint. Despite this scaling difference, both methods identify the same variables as the primary drivers of their first component all price-level features. The convergence of both methods on the same underlying structure provides strong evidence that the identified components reflect genuine market dynamics rather than an algorithmic artefact.

6. Sensitivity Analysis

To test robustness, PCA was re-run under three variance thresholds:

- 80% threshold: 3 components retained, explaining 86.4% of variance.
- 90% threshold: 4 components retained, explaining 90.6% of variance (primary analysis).
- 95% threshold: 5 components retained, explaining 96.7% of variance.

Critically, the PC1 loadings were identical across the 80% and 90% thresholds (maximum absolute difference = 0.0000), confirming that the interpretation of the dominant price-level component is completely stable. The choice of threshold only affects how many additional lower-variance components are retained, not the structure of the primary component.

7. Critical Discussion

Several limitations are important to acknowledge:

- **Linearity assumption:** PCA assumes linear relationships between variables. Financial markets exhibit non-linear dynamics such as volatility clustering and regime shifts. Kernel PCA or t-SNE could better handle such structure.
- **Non-stationarity:** Price-based features (Open, Close, EMA, SMA) are non-stationary time series. PC1 captures price magnitude rather than market dynamics. Using log-returns or differenced series would produce a more economically informative decomposition.
- **Temporal structure ignored:** PCA treats each observation as independent, violating the autocorrelation inherent in daily stock data. This may produce overoptimistic variance-explained estimates.
- **Post-hoc interpretation:** The financial labels assigned to each component are applied after the analysis and are not guaranteed to remain stable across different time periods or market regimes.
- **Multicollinearity:** The high correlation between SMA, EMA, Bollinger Bands, and OHLC prices means PC1 is essentially a redundant price-level index. Feature engineering replacing price levels with log-returns could produce a more informative PC1.
- **Outlier influence:** PCA is sensitive to extreme values. Outliers in Volume and Daily_Return were retained intentionally to preserve genuine market events, but their influence on component directions remains a limitation.

8. Conclusion

This analysis applied PCA to 16 numeric features derived from FAANG stock trading data. Four principal components were retained based on the Kaiser criterion and the 90% variance threshold, collectively explaining 90.6% of total variance with a reconstruction MSE of 0.069.

The four components were interpreted as: a Market Price Level factor (PC1, 63.6%), a Momentum/Oscillator factor (PC2, 14.1%), a Volume/Volatility factor (PC3, 6.6%), and a Residual Return factor (PC4, 6.3%). These align with three fundamental concepts in technical analysis - trend, momentum, and volatility, confirming that PCA recovers financially meaningful structure from the data.

Comparison with Factor Analysis showed consistent identification of the same dominant variables. Sensitivity analysis confirmed that the PC1 loading structure is perfectly stable across variance thresholds. Together, these results confirm that PCA extracts distinct, financially interpretable, and robust latent dimensions from FAANG stock technical indicators.

9. References

- [1] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>
- [2] M. S. Bartlett, "The effect of standardization on a χ^2 approximation in factor analysis," *Biometrika*, vol. 38, no. 3/4, pp. 337–344, 1951. [Online]. Available: <https://www.jstor.org/stable/2332588>
- [3] H. F. Kaiser, "An index of factorial simplicity," *Psychometrika*, vol. 39, no. 1, pp. 31–36, 1974. [Online]. Available: <https://doi.org/10.1007/BF02291575>
- [4] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002. [Online]. Available: <https://link.springer.com/book/10.1007/b98835>
- [5] J. J. Murphy, *Technical Analysis of the Financial Markets*. New York, NY, USA: New York Institute of Finance, 1999. [Online]. Available: <https://archive.org/details/john-j-murphy-technical-analysis-of-the-financial-markets/page/n9/mode/2up>
- [6] L. R. Fabrigar, D. T. Wegener, R. C. MacCallum, and E. J. Strahan, "Evaluating the use of exploratory factor analysis in psychological research," *Psychol. Methods*, vol. 4, no. 3, pp. 272–299, 1999. [Online]. Available: <https://doi.org/10.1037/1082-989X.4.3.272>

Appendix

```
# — Imports —
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, FactorAnalysis
from factor_analyzer.factor_analyzer import (
    calculate_bartlett_sphericity, calculate_kmo
)

from scipy import stats

np.random.seed(42)
sns.set_theme(style="whitegrid")

# — 1. Load Data —
df = pd.read_csv("faang_stock_prices.csv")
print(df.shape)  # (14964, 19)

# — 2. Missing Values —
print(df.isnull().sum())
df = df.dropna().reset_index(drop=True)

# — 3. Feature Selection —
# Exclude future target, date, and ticker identifier
df_numeric = df.drop(columns=["Date", "Ticker", "Next_Day_Close"])

# — 4. Outlier Detection (Z-Score) —
z_scores = pd.DataFrame(
    np.abs(stats.zscore(df_numeric, nan_policy="omit")),
    columns=df_numeric.columns, index=df_numeric.index
)

outlier_mask = (z_scores > 3).any(axis=1)
```

```

print(f"Rows with |Z|>3: {outlier_mask.sum()}
({outlier_mask.sum()/len(df_numeric)*100:.2f}%)")

# Outliers retained – represent genuine market events

# Boxplots of most affected features
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df_numeric["Volume"].plot(kind="box", ax=axes[0], title="Volume")
df_numeric["Daily_Return"].plot(kind="box", ax=axes[1], title="Daily_Return")
plt.suptitle("Boxplots – Features Most Sensitive to Outliers")
plt.tight_layout(); plt.show()

# — 5. Correlation Matrix —
plt.figure(figsize=(12, 9))
sns.heatmap(df_numeric.corr(), cmap="coolwarm", center=0,
            linewidths=0.3, annot=True, fmt=".2f", annot_kws={"size": 7})
plt.title("Correlation Matrix of Input Features")
plt.tight_layout(); plt.show()

# — 6. PCA Suitability —
corr_matrix = df_numeric.corr()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr = upper.stack()[upper.stack().abs() > 0.95]
cols_to_drop = list(set([col for col, _ in high_corr.index]))
df_clean = df_numeric.drop(columns=cols_to_drop)

chi2, p = calculate_bartlett_sphericity(df_clean)
print(f"Bartlett p = {p:.4e}")

kmo_all, kmo_model = calculate_kmo(df_clean)
print(f"KMO = {kmo_model:.4f}")

# — 7. Standardise —
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_numeric)

# — 8. Full PCA for scree plot —

```

```

pca_full = PCA()
pca_full.fit(X_scaled)
eigenvalues = pca_full.explained_variance_
explained    = pca_full.explained_variance_ratio_

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, len(eigenvalues)+1), eigenvalues, color="steelblue",
alpha=0.7)

axes[0].plot(range(1, len(eigenvalues)+1), eigenvalues, marker="o",
color="steelblue")

axes[0].axhline(y=1, linestyle="--", color="red", label="Kaiser ( $\lambda=1$ )")
axes[0].set_title("Scree Plot"); axes[0].legend()

axes[1].plot(range(1, len(explained)+1), np.cumsum(explained)*100, marker="o",
color="teal")

axes[1].axhline(90, linestyle="--", color="red", label="90% threshold")
axes[1].set_title("Cumulative Variance Explained"); axes[1].legend()
plt.tight_layout(); plt.show()

# — 9. Final PCA (4 components) —
pca = PCA(n_components=0.9)    # retains components until 90% variance is reached
X_pca = pca.fit_transform(X_scaled)
print(f"Components retained: {pca.n_components_}")

loadings = pd.DataFrame(
    pca.components_, columns=df_numeric.columns,
    index=[f"PC{i+1}" for i in range(pca.n_components_)]
)

# — 10. Loading Plots —
n_pcs = pca.n_components_
fig, axes = plt.subplots(1, n_pcs, figsize=(6*n_pcs, 6))
for i, ax in enumerate(axes):
    vals = loadings.iloc[i].sort_values()
    colours = ["tomato" if v < 0 else "steelblue" for v in vals]
    vals.plot(kind="barh", ax=ax, color=colours)
    ax.axvline(0, color="black", linewidth=0.8)
    ax.set_title(f"PC{i+1} ({pca.explained_variance_ratio_[i]*100:.1f}%)")

```

```

plt.tight_layout(); plt.show()

# — 11. Score Plots —

tickers = df["Ticker"].astype("category")
ticker_codes = tickers.cat.codes
ticker_labels = tickers.cat.categories
unique_codes = sorted(ticker_codes.unique())

def make_legend(unique_codes, ticker_labels):
    return [plt.Line2D([0],[0], marker="o", color="w",
                      markerfacecolor=plt.cm.viridis(c/max(unique_codes)),
                      markersize=8, label=ticker_labels[c]) for c in unique_codes]

pairs = [(0,1),(1,2)]
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
for ax, (xi, yi) in zip(axes, pairs):
    ax.scatter(X_pca[:,xi], X_pca[:,yi], c=ticker_codes, cmap="viridis", alpha=0.6,
s=15)
    ax.legend(handles=make_legend(unique_codes, ticker_labels), title="Ticker")
    ax.set_xlabel(f"PC{xi+1} ({pca.explained_variance_ratio_[xi]*100:.1f}%)")
    ax.set_ylabel(f"PC{yi+1} ({pca.explained_variance_ratio_[yi]*100:.1f}%)")
    ax.set_title(f"Score Plot: PC{xi+1} vs PC{yi+1}")
plt.tight_layout(); plt.show()

# — 12. Biplots —

score_scale = np.abs(X_pca[:, :2]).mean() * 2.5
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
for ax, (xi, yi) in zip(axes, pairs):
    ax.scatter(X_pca[:,xi], X_pca[:,yi], alpha=0.3, s=10, color="steelblue")
    for feature in df_numeric.columns:
        x = loadings.loc[f"PC{xi+1}", feature] * score_scale
        y = loadings.loc[f"PC{yi+1}", feature] * score_scale
        ax.arrow(0,0,x,y, color="red", alpha=0.75,
                head_width=score_scale*0.02, head_length=score_scale*0.015,
                length_includes_head=True)
        ax.text(x*1.13, y*1.13, feature, fontsize=7.5, color="darkred",
ha="center")

```

```

    ax.set_title(f"Biplot: PC{xi+1} vs PC{yi+1}")
plt.tight_layout(); plt.show()

# — 13. Reconstruction Error —
X_reconstructed = pca.inverse_transform(X_pca)
mse = np.mean((X_scaled - X_reconstructed)**2)
print(f"Reconstruction MSE: {mse:.6f}")

# — 14. Factor Analysis Comparison —
fa = FactorAnalysis(n_components=pca.n_components_)
X_fa = fa.fit_transform(X_scaled)

fa_loadings = pd.DataFrame(
    fa.components_, columns=df_numeric.columns,
    index=[f"FA{i+1}" for i in range(pca.n_components_)]
)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
for ax, (scores, label) in zip(axes, [(X_pca, "PCA Scores"), (X_fa, "FA Scores")]):
    ax.scatter(scores[:,0], scores[:,1], c=ticker_codes, cmap="viridis", alpha=0.6,
s=15)
    ax.legend(handles=make_legend(unique_codes, ticker_labels), title="Ticker")
    ax.set_xlabel("Component 1"); ax.set_ylabel("Component 2"); ax.set_title(label)
plt.suptitle("PCA vs FA Score Plots"); plt.tight_layout(); plt.show()

# — 15. Sensitivity Analysis —
for t in [0.80, 0.90, 0.95]:
    m = PCA(n_components=t).fit(X_scaled)
    print(f"{int(t*100)}%: {m.n_components_} components,
{m.explained_variance_ratio_.sum()*100:.2f}%")

# PC1 loading stability across thresholds
pca_80 = PCA(n_components=0.80).fit(X_scaled)
pca_90 = PCA(n_components=0.90).fit(X_scaled)
diff = np.abs(pca_80.components_[0] - pca_90.components_[0])
print(f"Max PC1 loading difference (80% vs 90%): {diff.max():.4f}")

```